Schema della Relazione

Paolo Danilo Secci

POLITECNICO
MILANO 1863

Progetto di Reti Logiche 2024-25
Ingegneria Informatica

Table of Contents:

Introduction

Objective:

The project's goal is to implement a Hardware module in VHDL that interfaces with a RAM memory and compiles with respect to the following specifications.

Specifications:

Byte by byte, the system reads a designated sequence from memory composed of 17 configuration bytes followed by K bytes of payload input. A differential filter is applied to each of the K input bytes, generating a corresponding output byte according to the system configuration.

Configurations:

The first 17 bytes of the designated sequence holds the configuration data. From these 17 bytes, we extract K, S, and C1-14. K, a 2-byte unsigned integer, denotes the number of bytes in the input payload; S, a byte with only its lead bit as a significant boolean, denotes whether the process is of Order 3 or Order 5; and C1-C14, an array of 14 bytes, denote the coefficients used in the differential filter.

The Differential Filter:

For each of the K input bytes in the sequence payload, the differential filter generates a corresponding output byte to be

written at the end of the sequence. The algorithm used to calculate the correct output for a given target byte of the input payload depends on S (Order 3 or Order 5), the Coefficients (from the configuration data), and the Neighbors (the bytes in the input payload that neighbor the target byte).

In the case of **Order 3** (S = 0), the algorithm uses 5 Coefficients, C2-C6, and collects 5 Neighbors, the target byte and 2 bytes on each side (or 0 if neighbor is out of the payload bounds).

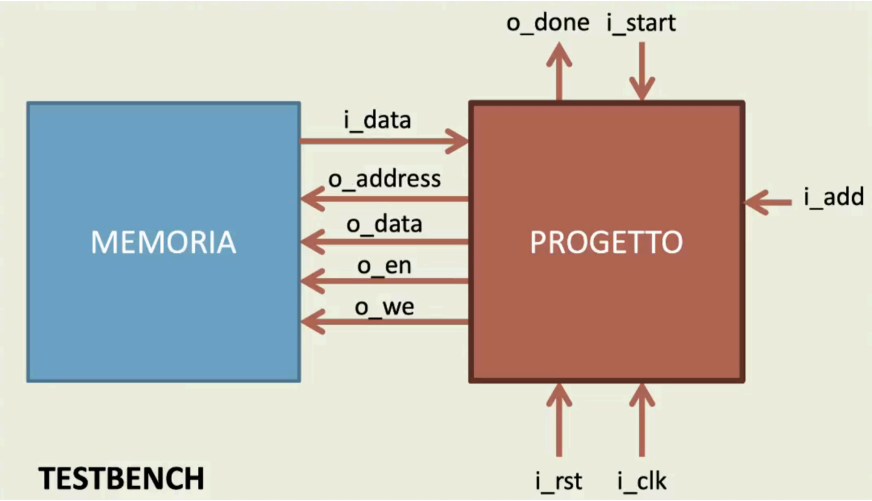
In the case of **Order 5** (S = 1), the algorithm uses 7 Coefficients, C8-C14, and collects 7 Neighbors, the target byte and 3 bytes on each side (or 0 if neighbor is out of the payload bounds).

Once the correct Coefficients and Neighbors are collected for the target byte, a two step function is implemented to generate the output byte. Step 1, the pns, pre-normalization sum, is calculated by the following formula: $pns = \Sigma(C[i] * N[i])$ where i traverses the 5 Coefficients and 5 Neighbors in Order 3, or it traverses the 7 Coefficients and 7 Neighbors in Order 5. Step 2, that pns is then normalized by the following formula: $byte\_out = (pns * n)$ using either n = 1/12 for Order 3 or n = 1/60 for Order 5. For this project, 1/12 is estimated using 1/16 + 1/64 + 1/256 + 1/1024; 1/60 is estimated using 1/64 + 1/1024.

Architecture

Interface:

The core module has three primary i/o connections: a one bit signal START, a 16-bit signal ADD, and a primary one bit output DONE. In addition, the module has a system-wide clock signal CLK and a system-wide reset signal RESET. All signals are synchronized to the rising edge of the CLK signal except for the asynchronous RESET signal.



In addition, the project module uses the following 5 i/o signals to interface with the memory: o_address, the output signal that carries a specific address to the memory; i_data, the input signal from the memory containing the data following a read request; o_data, the output signal to the memory that carries the data to be written subsequently; o_en, the “enable” signal that must be sent to the memory to enable both reading and writing; and o_we, the “write enable” signal that must be sent to the memory to write (which must be off to read).

Internal Signals:

The internal signals used in the VHDL module to implement the algorithm are the following: state, of type state_type containing the names of each FSM state; K, a 16-bit unsigned length; S, 1-bit selector (0 for Order 3, 1 for Order 5); c_array, a 14-byte coefficients array indexed from 0 to 13 (2's Complement); n_array: a 7-byte neighbors array indexed from -3 to 3 (2's Complement); byte_out, 18-bits used to calculate the pre-normalized sum (max: 81920) and to store the byte_out; cc, the "current counter" tracking the bytes in the sequence being processed; ca, the "current address" (16-bit); csa: the "current starting address" (16-bit); and ni, the neighbor index used to traverse the neighbors array.

FSM Design:

To properly execute the sequence processing, the FSM is designed in 9 states. The states are the following:

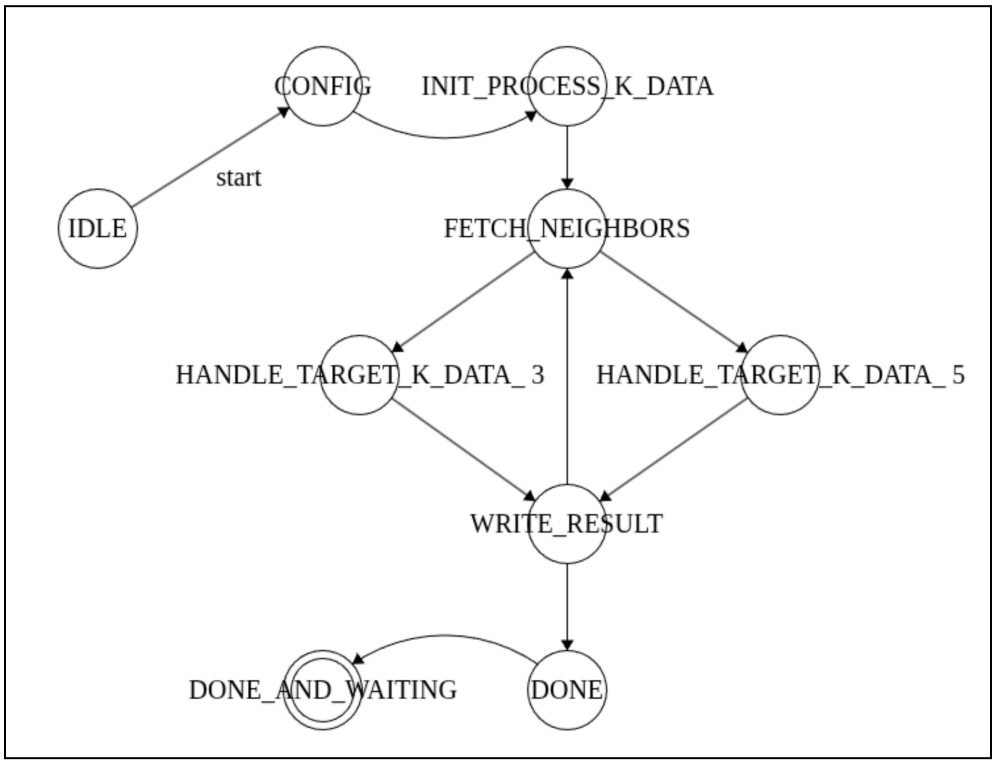
IDLE
CONFIG
INIT_PROCESS_K_DATA
FETCH_NEIGHBORS
HANDLE_TARGET_K_DATA_3
HANDLE_TARGET_K_DATA_5
WRITE_RESULT
DONE
DONE_AND_WAITING.

Starting with an IDLE state, the FSM loops idly and awaits the start signal to kick off the process. Upon receiving the start signal, the FSM goes into the CONFIG state where it requests and reads the 17 configuration bytes that determine key distinctions in the FSM's future state flow. Next the FSM goes into the INIT_PROCESS_K_DATA which will set up the first of the K payload input bytes to be processed either as Order 3 or Order 5 (the others will follow suit).

Then the FSM enters the main loop which traverses the K payload bytes. For each target byte, we start in the FETCH_NEIGHBORS state, which collects the bytes neighboring the target byte. Once the neighbors have been collected, the FSM moves into a state to implement the differential filter, either HANDLE_TARGET_K_DATA_3 or HANDLE_TARGET_K_DATA_5. In these states, the FSM applies the differential filter as a two-step function to calculate the correct byte_out for Order 3 and Order 5 respectively. The

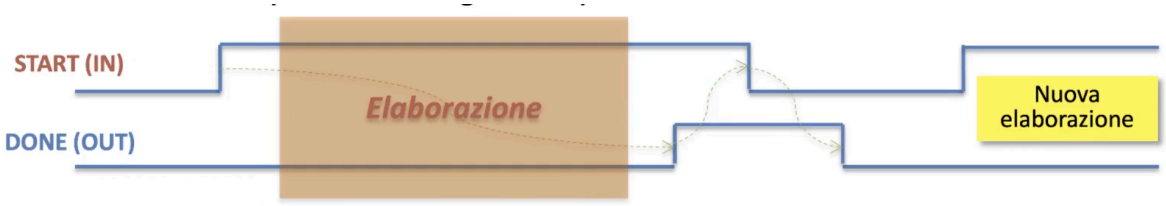
FSM proceeds to a WRITE_RESULT state which sends to memory a request to write the calculated byte_out to the corresponding output address.

After writing the byte_out, the FSM determines if it has finished processing all K of the input bytes, or if it should continue the main loop and begin in FETCH_NEIGHBORS state for the next target byte. If the FSM has finished processing all K bytes of the input payload, then it proceeds to the DONE state, where the FSM closes up the system permissions and sends the done signal. Finally, the FSM goes to a DONE_AND_WAITING state where it waits for the memory to process the done signal and close the start signal.



Synchronization:

At any of the states, an asynchronous reset signal could trigger the FSM to go right back to the IDLE state. Given that the process is not interrupted by an asynchronous signal, each of the turns of the machine happen on the falling edge of the clock. To kick off the process out of the idle state, the start signal 1 is required, and as long as that start stays high, the reset signal doesn't change, and the done signal is not sent, the process will continue the elaboration of the given sequence. At the end of the elaboration, the done signal must be sent from the FSM, and the FSM must then wait for the lowering of the start signal to officially close and lower the done signal.



Experiment Results

Synthesis:

The following synthesis utilizability report was generated using Vivado 2025.1 targeting the FPGA xc7vx485tffg1157-1.

```
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved. Copyright 2022-2025 Advanced
Micro Devices, Inc. All Rights Reserved.
------------------------------------------------------------------------------------
------------------------------------------------------------
| Tool Version : Vivado v.2025.1 (lin64) Build 6140274 Wed May 21 22:58:25 MDT 2025
| Date         : Sun Sep 14 21:50:55 2025
| Host         : ****** running 64-bit Ubuntu 24.04.2 LTS
| Command      : report_utilization -file project_reti_logiche_utilization_synth.rpt
-pb project_reti_logiche_utilization_synth.pb
| Design       : project_reti_logiche
| Device       : xc7vx485tffg1157-1
| Speed File   : -1
| Design State : Synthesized
------------------------------------------------------------------------------------
------------------------------------------------------------

Utilization Design Information

Table of Contents
-----------------
1. Slice Logic
1.1 Summary of Registers by Type
2. Memory
3. DSP
4. IO and GT Specific
5. Clocking
6. Specific Feature
7. Primitives
8. Black Boxes
9. Instantiated Netlists

1. Slice Logic
--------------

+-------------------------+------+-------+------------+-----------+-------+
|        Site Type        | Used | Fixed | Prohibited | Available | Util% |
+-------------------------+------+-------+------------+-----------+-------+
| Slice LUTs*             | 1437 |     0 |          0 |    303600 |  0.47 |
|   LUT as Logic          | 1437 |     0 |          0 |    303600 |  0.47 |
|   LUT as Memory         |    0 |     0 |          0 |    130800 |  0.00 |
| Slice Registers         |  270 |     0 |          0 |    607200 |  0.04 |
|   Register as Flip Flop |  270 |     0 |          0 |    607200 |  0.04 |
```

Register as Latch	0	0	0	607200	0.00
F7 Muxes	0	0	0	151800	0.00
F8 Muxes	0	0	0	75900	0.00

* Warning! The Final LUT count, after physical optimizations and full implementation, is typically lower. Run opt_design after synthesis, if not already completed, for a more realistic count.
Warning! LUT value is adjusted to account for LUT combining.
Warning! For any ECO changes, please run place_design if there are unplaced instances

1.1 Summary of Registers by Type

Total	Clock Enable	Synchronous	Asynchronous
0	_	-	-
0	_	-	Set
0	_	-	Reset
0	_	Set	-
0	_	Reset	-
0	Yes	-	-
0	Yes	-	Set
252	Yes	-	Reset
0	Yes	Set	-
18	Yes	Reset	-

2. Memory

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	0	0	0	1030	0.00
RAMB36/FIFO*	0	0	0	1030	0.00
RAMB18	0	0	0	2060	0.00

* Note: Each Block RAM Tile only has one FIFO logic available and therefore can accommodate only one FIFO36E1 or one FIFO18E1. However, if a FIFO18E1 occupies a Block RAM Tile, that tile can still accommodate a RAMB18E1

3. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	0	0	0	2800	0.00

4. IO and GT Specific

Site Type	Used	Fixed	Prohibited	Available	Util%
Bonded IOB	54	0	0	600	9.00
Bonded IPADs	0	0	0	62	0.00
Bonded OPADs	0	0	0	40	0.00
PHY_CONTROL	0	0	0	14	0.00
PHASER_REF	0	0	0	14	0.00
OUT_FIFO	0	0	0	56	0.00
IN_FIFO	0	0	0	56	0.00
IDELAYCTRL	0	0	0	14	0.00
IBUFDS	0	0	0	576	0.00
GTXE2_COMMON	0	0	0	5	0.00
GTXE2_CHANNEL	0	0	0	20	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	0	56	0.00
PHASER_IN/PHASER_IN_PHY	0	0	0	56	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	0	700	0.00
ODELAYE2/ODELAYE2_FINEDELAY	0	0	0	700	0.00
IBUFDS_GTE2	0	0	0	10	0.00
ILOGIC	0	0	0	600	0.00
OLOGIC	0	0	0	600	0.00

5. Clocking

Site Type	Used	Fixed	Prohibited	Available	Util%
BUFGCTRL	1	0	0	32	3.13
BUFIO	0	0	0	56	0.00
MMCME2_ADV	0	0	0	14	0.00
PLLE2_ADV	0	0	0	14	0.00
BUFMRCE	0	0	0	28	0.00
BUFHCE	0	0	0	168	0.00
BUFR	0	0	0	56	0.00

6. Specific Feature

Site Type	Used	Fixed	Prohibited	Available	Util%
BSCANE2	0	0	0	4	0.00
CAPTUREE2	0	0	0	1	0.00
DNA_PORT	0	0	0	1	0.00
EFUSE_USR	0	0	0	1	0.00
FRAME_ECCE2	0	0	0	1	0.00
ICAPE2	0	0	0	2	0.00
PCIE_2_1	0	0	0	4	0.00

| STARTUPE2 | 0 | 0 | 0 | 1 | 0.00 |
| XADC | 0 | 0 | 0 | 1 | 0.00 |

7. Primitives

Ref Name	Used	Functional Category
LUT2	586	LUT
LUT6	525	LUT
LUT4	446	LUT
CARRY4	284	CarryLogic
FDCE	252	Flop & Latch
LUT5	134	LUT
LUT3	107	LUT
LUT1	80	LUT
OBUF	27	IO
IBUF	27	IO
FDRE	18	Flop & Latch
BUFG	1	Clock

8. Black Boxes

Ref Name	Used

9. Instantiated Netlists

Ref Name	Used

Simulation:

In order to test the correctness of the FSM and VHDL module, I elaborated on Professor Palermo's TestBench. Specifically **paolo_tb.vhd** verifies the VHDL digital filter design by simulating two scenarios (Order 3 and Order 5) to ensure correct functionality across different configurations.

The **Order 3 Scenario** sets K to 24, S to 0, loads C1-C14 as [0,-1,8,0,-8,1,0,1,-9,45,0,-45,9,-1] into RAM(1234:1250), and **Order 5 Scenario** sets K to 47, S to 1, loads C1-C14 as [0,0,0,0,0,0,0,1,-9,45,0,-45,9,-1] into RAM(2025:2041). Then each scenario has a precalculated input K bytes with the corresponding output K bytes. The testbench initializes the design with a reset, sets the start address, and triggers the filter. After the process runs through the sequence, several "Assertions" check that

1. RAM outputs match the correct Scenario 1 output,
2. tb_done is 0 during reset and before start, and
3. tb_o_mem_en and tb_o_mem_we are 0 after tb_done=1.

The testbench uses a 20 ns clock and a memory_control signal to switch RAM access between testbench initialization and the design, ensuring proper data loading and verification. All simulations (pre-synthesis, post-synthesis functional, and timing) pass at 20.73 µs logging "Simulation Ended! ALL TESTS PASSED," confirming the design's success.

Conclusion:

The detailed VHDL digital filter processes K input bytes with a differential filter of Order 3 or 5, by reading data from RAM, processing the data using a 9-state FSM, and writing results back to RAM. The FSM is synchronized to a 20 ns clock with asynchronous reset.

Using Vivado to ensure the VHDL module's proper implementation, pre-synthesis, post-synthesis functional, and post-synthesis timing simulations passed a series of TestBench scenarios at 20.73 µs, confirming functional correctness.

Lastly, the Synthesis Report on FPGA xc7vx485tffg1157-1 shows efficient resource usage with 1437 LUTs (0.47%), 270 Flip-Flops (0.04%), 0 DSPs, and 0 Block RAM.